

Computación de Alto Rendimiento (HPC)

Caso de Estudio 1

Aplicación de concurrencia y paralelismo con hilos y procesos a la multiplicación de matrices

José Alejandro Quintero Gahona

[Nombre integrante 2]

[Nombre integrante 3]

Docente: [Nombre del profesor]

Asignatura: Computación de Alto Rendimiento

[Fecha de entrega]

Índice

1. Introducción	3
2. Objetivos	3
3. Marco teórico	3
3.1. Wall clock y tiempo de CPU	3
3.2. Speedup	4
3.3. Por qué se repite el experimento 10 veces	4
3.4. El orden de los bucles importa	5
3.5. Hilos POSIX (pthreads)	5
3.6. Procesos POSIX y <code>fork()</code>	5
3.7. Memoria compartida POSIX	5
3.8. Quantum y planificación	6
4. Arquitectura del código	6
4.1. Versión secuencial	6
4.2. Versión con pthreads	6
4.3. Versión con fork y memoria compartida	7
4.4. Estrategia de partición	7
4.5. Por qué no se necesita sincronización adicional	7
5. Diseño experimental	7
5.1. Hardware	7
5.2. Variables	8
5.3. Metodología: 10 corridas por configuración	8
6. Resultados	8
6.1. Tiempos medidos con pthreads	8
6.2. Tiempos medidos con fork	8
6.3. Speedup con pthreads	9
6.4. Speedup con fork	9
6.5. Comparativa pthreads vs fork	9
6.6. Gráficas	9
6.7. Outliers detectados	10
7. Análisis de resultados	11
7.1. Fork gana en problemas chicos	11
7.2. Pthreads gana en problemas grandes	11
7.3. El punto de cruce está en $N = 2000$	11
7.4. Los outliers y la robustez de la mediana	12

8. Conclusiones	12
8.1. Logros del trabajo	12
8.2. Hallazgos principales	12
8.3. Limitaciones	13
8.4. Trabajo futuro	13
9. Referencias	13

1. Introducción

Este documento presenta el desarrollo del caso de estudio 1 de la materia de Computación de Alto Rendimiento (HPC). El objetivo es aplicar concurrencia y paralelismo a bajo nivel a un programa de multiplicación de matrices que se había implementado previamente en la versión inicial del proyecto.

A lo largo del trabajo se construyeron dos versiones paralelas del programa: una usando **POSIX Threads (pthreads)** y otra usando **procesos POSIX con fork() y memoria compartida**. Ambas se comparan con la versión secuencial de referencia.

El resto del documento está organizado así: la sección 2 enumera los objetivos; la sección 3 presenta los conceptos teóricos mínimos necesarios; la sección 4 describe la arquitectura del código; la sección 5 explica el diseño experimental; la sección 6 muestra los resultados; la sección 7 los analiza; y la sección 8 cierra con las conclusiones.

2. Objetivos

Objetivo general

Implementar dos versiones paralelas del programa de multiplicación de matrices, una con hilos y otra con procesos, usando la API POSIX directamente (sin frameworks de más alto nivel), y comparar su desempeño con la versión secuencial.

Objetivos específicos

- Implementar la versión paralela con **POSIX Threads**, compartiendo memoria entre los hilos de forma implícita.
- Implementar la versión paralela con **procesos** usando **fork()**, con comunicación entre procesos mediante memoria compartida POSIX (**shm_open + mmap**).
- Diseñar un experimento que varíe el tamaño de la matriz y la cantidad de hilos/procesos, repitiendo cada configuración 10 veces para tener resultados estadísticamente confiables.
- Calcular el **speedup** de cada técnica y comparar pthread contra fork para identificar en qué condiciones una le gana a la otra.

3. Marco teórico

En esta sección se introducen los conceptos mínimos que se necesitan para entender el resto del documento. Se evitan las fórmulas largas y los desarrollos matemáticos; el objetivo es tener un vocabulario común.

3.1. Wall clock y tiempo de CPU

En cualquier sistema operativo hay dos formas de medir cuánto tarda un programa:

- **Wall clock:** es el tiempo real que pasa desde que el programa arranca hasta que termina. Es lo que un usuario percibe como “cuánto tardó”.
- **Tiempo de CPU:** es el tiempo que la CPU pasó efectivamente ejecutando instrucciones de nuestro programa, sin contar esperas.

Durante la materia, el profesor indicó que la métrica correcta para comparar rendimiento entre programas es el **wall clock**, porque refleja la experiencia real del usuario. En este trabajo se mide con la función `clock_gettime(CLOCK_MONOTONIC_RAW, ...)`, que es la opción más estable para benchmarking: no se ve afectada por los ajustes automáticos que el sistema hace al reloj (NTP).

Para todas las mediciones se cronometra **solo el trabajo de multiplicación**. La reserva de memoria, la generación de las matrices aleatorias y la impresión de resultados quedan fuera del tiempo medido, porque no son el trabajo que estamos analizando.

3.2. Speedup

El *speedup* es la métrica estándar para medir cuánto más rápido es un programa paralelo respecto al secuencial. Se calcula como:

$$S_T = \frac{T_{\text{secuencial}}}{T_{\text{paralelo}, T}}$$

donde T es la cantidad de hilos o procesos. El mejor caso posible con T unidades es $S_T = T$ (lo que se llama *speedup lineal*); en la práctica siempre es menor, por el *overhead* de crear los hilos/procesos, la sincronización y la contención de recursos.

3.3. Por qué se repite el experimento 10 veces

Una sola medición no es estadísticamente confiable: el sistema operativo puede tener otras tareas corriendo, una preempción puede caer justo en nuestra corrida, o un proceso del sistema puede usar la CPU en el momento crítico. Para aislar este **ruido del sistema**, se repite el experimento 10 veces por configuración y se calcula:

- **Media** aritmética: el promedio de las 10 mediciones.
- **Mediana:** el valor central al ordenar las 10 mediciones. Es **robusta** frente a valores atípicos: si una sola medición es muy rara, no distorsiona el resultado.
- **Desviación estándar** (σ): qué tanto se dispersan las mediciones.

En este trabajo se usa la **mediana** como medida principal del speedup.

3.4. El orden de los bucles importa

Un detalle del script que ejecuta el experimento: para cada repetición se recorren **todos** los tamaños de matriz antes de pasar a la siguiente repetición. Es decir, el orden es **for cada_repetición: for cada_N: medir**, y no al revés.

El profesor explicó que si las 10 repeticiones de un mismo tamaño se hicieran seguidas, los datos quedarían **calientes** en las memorias rápidas del procesador (caché) y el sistema no se molestaría en traerlos desde la memoria RAM. La medición saldría **artificialmente rápida**. Al intercalar tamaños entre repeticiones, cada medición se ve forzada a trabajar con datos nuevos en RAM, dando un resultado más realista.

3.5. Hilos POSIX (pthreads)

Un **hilo** (*thread*) es un flujo de ejecución que vive dentro de un proceso. Varios hilos de un mismo proceso comparten la memoria, los archivos abiertos y otros recursos, pero cada hilo tiene su propia pila y registros.

La API estándar POSIX para crear y manipular hilos se llama **pthreads**. En este trabajo usamos dos funciones:

- `pthread_create(&tid, NULL, funcion, arg)`: crea un nuevo hilo que ejecuta `funcion(arg)`. Retorna inmediatamente.
- `pthread_join(tid, NULL)`: bloquea al hilo que llama hasta que el hilo `tid` termine.

3.6. Procesos POSIX y fork()

Un **proceso** es una unidad de ejecución con su propio espacio de direcciones. Crear un proceso es más caro que crear un hilo (el sistema tiene que copiar tablas de memoria, asignar un identificador nuevo, etc.), pero a cambio cada proceso está completamente aislado de los demás.

En este trabajo usamos la llamada al sistema `fork()`, que crea un proceso hijo idéntico al padre. Después del `fork()`:

- El **padre** recibe como resultado el PID (identificador) del hijo creado.
- El **hijo** recibe como resultado 0.
- Si `fork()` falla, retorna un valor negativo.

Para esperar a que el hijo termine, el padre usa `waitpid(pid, NULL, 0)`.

3.7. Memoria compartida POSIX

A diferencia de los hilos, los procesos **no comparten memoria** por defecto. Si dos procesos necesitan ver los mismos datos, hay que usar un mecanismo explícito. En este trabajo usamos la API de **memoria compartida POSIX**:

- `shm_open(nombre, ...)`: crea o abre una región de memoria identificada por un nombre (similar a un archivo en `/dev/shm`).
- `ftruncate(fd, tamaño)`: fija el tamaño de la región.
- `mmap(NULL, tamaño, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0)`: mapea la región en la memoria del proceso. Con `MAP_SHARED`, las escrituras de un proceso son visibles para todos los demás que mapearon la misma región.
- `shm_unlink(nombre)`: elimina la región cuando ya no se necesita.

Cada matriz (A, B, C) tiene su propia región de memoria compartida. El padre las llena con los datos, los hijos las ven y escriben en ellas, y al final todos leen los resultados desde la misma memoria.

3.8. Quantum y planificación

Un proceso **no se ejecuta de corrido hasta terminar**. El sistema operativo le asigna un tiempo fijo llamado **quantum** (del orden de 10 ms). Cuando se acaba el quantum, el sistema pone al proceso en una cola de “listos” y le da el CPU a otro. El profesor explicó que el quantum es **por proceso**, no por hilo: si un proceso tiene 100 hilos, el quantum se reparte entre los 100. Por eso conviene crear hilos “con medida”, no a lo loco.

4. Arquitectura del código

El proyecto se compone de **tres versiones** del mismo programa, más una serie de módulos compartidos y los scripts que automatizan el experimento. La figura 1 muestra el árbol completo de archivos.

4.1. Versión secuencial

Es el programa base sobre el que se construyen las versiones paralelas. Recibe como argumentos el tamaño de la matriz `N` y un valor máximo, reserva memoria para tres matrices, las llena con números aleatorios, ejecuta el triple bucle de multiplicación y mide el tiempo. El código vive en `src/matriz_monotonic.c`.

4.2. Versión con pthreads

Variante que reparte las filas de la matriz resultado entre varios hilos usando la API POSIX Threads. Los hilos se crean con `pthread_create` y el programa principal espera con `pthread_join`. El código de la paralelización vive en `modules/pthread_mult.c`.

4.3. Versión con fork y memoria compartida

Variante que crea procesos con `fork()` en lugar de hilos. Como cada proceso tiene su propio espacio de direcciones, las matrices se crean en **memoria compartida POSIX** para que padre e hijos vean los mismos datos. El padre espera a cada hijo con `waitpid()`. El código de la paralelización vive en `modules/fork_mult.c` y la gestión de la memoria compartida en `modules/matrix_shm.c`.

4.4. Estrategia de partición

Las tres versiones paralelas usan la **misma estrategia**: repartir las filas de la matriz resultado C entre los trabajadores. Cada uno calcula las filas que le corresponden y solo escribe en esa parte, así que no hay conflicto entre ellos.

El reparto es equitativo: si N es divisible por la cantidad de hilos/procesos, cada uno recibe N/T filas. Si no, los primeros reciben una fila extra para balancear la carga. El siguiente ejemplo muestra la partición para $N = 1000$ y $T = 4$:

Hilo / Proceso	Filas asignadas
0	[0, 250)
1	[250, 500)
2	[500, 750)
3	[750, 1000)

4.5. Por qué no se necesita sincronización adicional

Como cada hilo o proceso escribe **exclusivamente** en su propio rango de filas de la matriz C , no hay condiciones de carrera: dos trabajadores nunca escriben sobre la misma posición. Los trabajadores leen de A y B sin escribir, lo cual es seguro. La única sincronización necesaria es al final: `pthread_join` en la versión pthreads, o `waitpid` en la versión fork, para que el programa principal espere a que todos terminen antes de mostrar el resultado.

5. Diseño experimental

Esta sección describe cómo se diseñó el experimento. El profesor hizo énfasis en que el diseño experimental es la parte más importante del documento, porque sin un buen diseño los números no significan nada.

5.1. Hardware

Todas las mediciones se realizaron en el mismo equipo, por los integrantes del grupo. Las características son:

- **CPU**: Intel Core i5-12450HX (12.^a generación).
- **Núcleos lógicos**: 12 (8 núcleos físicos con *hyperthreading* activo).

- **Sistema operativo:** Linux.
- **Compilador:** gcc versión reciente con flags `-Wall -Wextra -O2`.

5.2. Variables

- **Tamaño de la matriz (N):** 500, 1000, 2000, 4000 y 8000. Se eligió un rango amplio con saltos de factor 2 para cubrir tanto problemas chicos (donde el paralelismo es marginal) como problemas grandes (donde el paralelismo muestra todo su potencial).
- **Cantidad de hilos o procesos (T):** 1 (secuencial), 2, 4, 8 y 16. Para la versión fork se saltea T=1 porque es esencialmente la misma versión secuencial que ya está medida.
- **Variable medida:** tiempo de la multiplicación en segundos, usando `clock_gettime(CLOCK_MONOTONIC)`.
- **Métrica calculada:** $speedup = \text{tiempo secuencial} / \text{tiempo paralelo}$.

En total se ejecutan **250 mediciones con pthreads** (5 tamaños $\times$ 5 configuraciones $\times$ 10 corridas) y **200 mediciones con fork** (5 $\times$ 4 $\times$ 10).

5.3. Metodología: 10 corridas por configuración

Para cada combinación de tamaño y cantidad de hilos/procesos se ejecutaron **10 corridas independientes** y se calculó la mediana, la media y la desviación estándar. El profesor insistió en que una sola medición no es confiable porque una sola corrida puede coincidir con un momento de ruido del sistema (otra tarea, una preempción, etc.). Repetir 10 veces y usar la mediana como estimador central reduce el impacto de esos picos.

Para la versión fork se midió el tiempo desde

6. Resultados

En esta sección se muestran los tiempos medidos y los speedup calculados. Los valores que se reportan son la **mediana** de las 10 corridas, porque es la medida más robusta frente a valores atípicos. Todos los tiempos están en segundos.

6.1. Tiempos medidos con pthreads

La tabla 1 muestra el tiempo de la multiplicación para la versión pthreads en segundos.

6.2. Tiempos medidos con fork

La tabla 2 muestra lo mismo para la versión fork. No se midió T=1 porque es esencialmente la versión secuencial ya medida arriba.

N	T=1	T=2	T=4	T=8	T=16
500	0.044	0.101	0.053	0.034	0.032
1000	0.423	0.753	0.398	0.266	0.204
2000	4.36	7.12	3.76	2.39	1.64
4000	75.65	68.22	40.30	29.68	24.29
8000	2441.6	690.9	416.9	354.1	401.5

Cuadro 1. Tiempos de ejecución con pthreads (segundos, mediana de 10 corridas).

N	T=2	T=4	T=8	T=16
500	0.029	0.022	0.015	0.015
1000	0.239	0.143	0.130	0.117
2000	4.86	2.36	1.98	3.01
4000	110.5	65.3	59.1	64.6
8000	1197.2	742.7	796.2	940.1

Cuadro 2. Tiempos de ejecución con fork (segundos, mediana de 10 corridas).

6.3. Speedup con pthreads

La tabla 3 muestra el *speedup* de pthreads calculado como tiempo secuencial / tiempo paralelo.

N	T=2	T=4	T=8	T=16
500	0.44	0.83	1.27	1.40
1000	0.56	1.06	1.59	2.07
2000	0.61	1.16	1.83	2.66
4000	1.11	1.88	2.55	3.11
8000	3.53	5.86	6.90	6.08

Cuadro 3. Speedup de pthreads.

6.4. Speedup con fork

La tabla 4 muestra el *speedup* de fork. La base secuencial usada es la misma que para pthreads (T=1 de la versión secuencial).

6.5. Comparativa pthreads vs fork

La tabla 5 compara directamente las dos técnicas. La columna “Mejor” indica qué técnica dio mayor speedup en esa combinación.

6.6. Gráficas

La figura 2 muestra el speedup de la versión pthreads en función del tamaño de la matriz. Se observa que el speedup crece con N y satura cerca de $N = 4000$.

N	T=2	T=4	T=8	T=16
500	1.54	1.99	2.89	2.88
1000	1.77	2.95	3.25	3.62
2000	0.90	1.85	2.20	1.45
4000	0.69	1.16	1.28	1.17
8000	2.04	3.29	3.07	2.60

Cuadro 4. Speedup de fork (comparado con el T=1 de pthreads).

N	T	pthreads	fork	Mejor
500	2	0.44	1.54	fork
500	4	0.83	1.99	fork
500	8	1.27	2.89	fork
500	16	1.40	2.88	fork
1000	2	0.56	1.77	fork
1000	4	1.06	2.95	fork
1000	8	1.59	3.25	fork
1000	16	2.07	3.62	fork
2000	2	0.61	0.90	fork
2000	4	1.16	1.85	fork
2000	8	1.83	2.20	fork
2000	16	2.66	1.45	pthreads
4000	2	1.11	0.69	pthreads
4000	4	1.88	1.16	pthreads
4000	8	2.55	1.28	pthreads
4000	16	3.11	1.17	pthreads
8000	2	3.53	2.04	pthreads
8000	4	5.86	3.29	pthreads
8000	8	6.90	3.07	pthreads
8000	16	6.08	2.60	pthreads

Cuadro 5. Comparativa de speedup: pthreads vs fork.

La figura 3 muestra lo mismo para la versión fork. La forma es similar, pero los valores son diferentes: en N chicos el speedup es más alto.

La figura 4 es la gráfica principal de este trabajo: muestra las 8 curvas (4 de pthreads y 4 de fork) en una sola imagen para facilitar la comparación directa.

6.7. Outliers detectados

Al revisar las 10 corridas individuales de cada configuración con fork, se identificaron cinco valores atípicos puntuales. Todos corresponden a la **corrida número 10** de las configuraciones con $T = 2$. La tabla 6 los resume.

Que todos los outliers caigan en la corrida 10 de las configuraciones con $T = 2$ sugiere que se debió a **ruido del sistema** en una ventana de tiempo específica (otra tarea puntual del sistema operativo), no a un problema del programa. El uso de la **mediana** como medida

Configuración	Outlier	Tiempo típico	Desviación
N=500, T=2, run=10	0.057 s	~0.029 s	casi 2×
N=1000, T=2, run=10	0.426 s	~0.244 s	80 % arriba
N=2000, T=2, run=10	9.84 s	~4.86 s	el doble
N=4000, T=2, run=10	152 s	~110 s	37 % arriba
N=8000, T=2, run=10	1327 s	~1197 s	11 % arriba

Cuadro 6. Valores atípicos puntuales en fork, todos en T=2 run=10.

central hace que estos picos no distorsionen el speedup reportado, mientras que la media se vería fuertemente afectada.

7. Análisis de resultados

7.1. Fork gana en problemas chicos

En las matrices pequeñas ($N = 500$ y $N = 1000$), la versión con `fork()` supera claramente a la versión con `pthread`s. Por ejemplo, con $N = 1000$ y 16 procesos fork logra un speedup de $3.62\times$, mientras que `pthread`s con 16 hilos solo llega a $2.07\times$.

Esto puede sonar contra-intuitivo, porque crear un proceso es más caro que crear un hilo. La explicación está en que para problemas chicos, **el tiempo total es muy pequeño** y la mayor parte de lo que se mide es *overhead*. El *overhead* de fork (crear el proceso, esperar, limpiar) se paga una sola vez por medición, así que pesa menos proporcionalmente. En cambio, el *overhead* de `pthread`s, aunque más liviano, se paga 16 veces (uno por cada hilo).

7.2. Pthreads gana en problemas grandes

En las matrices grandes ($N = 4000$ y $N = 8000$), la tendencia se invierte. Con $N = 8000$ y 8 hilos, `pthread`s alcanza un speedup de $6.90\times$, mientras que fork con 8 procesos solo llega a $3.07\times$: `pthread`s es más del doble de rápido.

La razón: cuando el problema es grande, el cómputo domina y el *overhead* pesa menos. Ahí se ve el costo real de las dos estrategias:

- **pthread**s: comparten memoria de forma **implícita**. No hay que configurar nada especial para que vean los mismos datos.
- **fork**: cada proceso tiene su propia imagen de memoria. Para compartir las matrices hay que pasar por `shm_open`, `mmap` y `shm_unlink`. Cuando N es grande y los datos son grandes, ese trabajo extra de configuración se nota.

7.3. El punto de cruce está en $N = 2000$

Mirando la tabla 5 y la figura 4, se ve que en $N = 2000$ las dos técnicas están prácticamente empatadas: en algunas configuraciones gana fork y en otras gana `pthread`s. Este es el **punto de cruce** entre las dos técnicas. Para matrices más chicas que $N = 2000$ conviene fork; para matrices más grandes, conviene `pthread`s.

7.4. Los outliers y la robustez de la mediana

El patrón de los outliers (todos en $T=2$, $\text{run}=10$) muestra por qué fue importante repetir 10 veces y usar la mediana. Si solo se hubiera medido una vez por configuración, algunos speedup habrían sido pesimistas por culpa de ese ruido.

El contraste es claro: la media es sensible a un valor extremo, mientras que la mediana lo ignora. Por ejemplo, en $N = 2000$ con $T = 2$:

- Media = 5.38 s
- Mediana = 4.86 s
- Diferencia = 11 % (el outlier de 9.84 s infla la media)

Si el speedup se hubiera calculado con la media, habría sido $3.28\times$; con la mediana, $3.53\times$. La diferencia no es dramática, pero confirma que la mediana es la elección correcta.

8. Conclusiones

8.1. Logros del trabajo

Se cumplió el objetivo general del caso de estudio: se implementaron dos versiones paralelas del programa de multiplicación de matrices, una con **POSIX Threads** y otra con **procesos + memoria compartida POSIX**, y se compararon con la versión secuencial usando la métrica estándar de speedup.

El diseño experimental fue riguroso: 10 corridas por configuración, 5 tamaños de matriz, hasta 5 cantidades de hilos/procesos, en total 450 mediciones. El script se diseñó de forma que se puede pausar y reanudar sin perder datos, lo cual fue clave para un experimento de varias horas.

8.2. Hallazgos principales

1. **No hay una técnica universalmente mejor:** la elección entre pthreads y fork depende del tamaño del problema. En este equipo, para matrices chicas ($N < 2000$) fork fue mejor; para matrices grandes ($N \geq 4000$) pthreads fue mejor.
2. **El speedup real está lejos del ideal.** El mejor caso fue $6.90\times$ con 8 hilos en $N = 8000$, mientras que el ideal sería $8\times$ con 8 hilos o $12\times$ con 12 procesos (recordando que la máquina tiene 12 núcleos lógicos). La diferencia se debe a la parte secuencial del programa, la contención por memoria y el *overhead* del sistema.
3. **Repetir el experimento 10 veces y usar la mediana fue clave:** se detectaron outliers puntuales debidos a ruido del sistema, y la mediana los ignoró de forma natural.
4. **El orden de los bucles en el script:** alternar tamaños entre repeticiones evitó que los datos se quedaran “calientes” en la caché del procesador, dando mediciones más realistas.

8.3. Limitaciones

- El speedup de fork se calculó contra el $T = 1$ de **pthread**s (no se midió fork $T = 1$ por separado). En teoría fork $T = 1$ debería dar un tiempo similar al secuencial de pthreads, pero no se verificó experimentalmente.
- La versión fork con $N = 8000$ usa mucha memoria: tres matrices de 256 MB cada una, más la imagen del proceso. En máquinas con menos RAM podría fallar.
- El experimento se hizo en un solo equipo. En otra máquina con distinto hardware (más o menos núcleos, diferente jerarquía de caché) los resultados podrían cambiar.

8.4. Trabajo futuro

Como continuación natural de este trabajo se podrían explorar:

- Implementar la versión con procesos midiendo también $T=1$, para tener un baseline propio de fork.
- Probar tamaños de matriz aún más grandes ($N = 16000$ o más) para ver si la tendencia de pthreads ganando se mantiene.
- Aplicar optimizaciones al algoritmo (caché blocking, orden de los bucles cambiado) y medir cómo afectan al speedup.
- Probar en una máquina con muchos más núcleos (32 o 64) para ver hasta dónde escala cada técnica.

9. Referencias

- **The Open Group Base Specifications Issue 7 (POSIX.1-2008)** — pthread_create, pthread_join, fork(), waitpid(), shm_open, mmap, clock_gettime. Documentación oficial de la API POSIX utilizada en este trabajo.
- **GNU C Library (glibc) Manual** — Documentación de las funciones de la biblioteca estándar de C, incluyendo la implementación de pthreads y memoria compartida en sistemas Linux.
- **Material del curso de Computación de Alto Rendimiento** — Presentaciones y grabaciones de clase utilizadas como referencia conceptual (wall clock vs tiempo de CPU, 10 corridas, orden de los bucles, quantum, memoria compartida).

```
├─ Makefile
├─ README.md
├─ experiments/
│   ├─ aggregate.py
│   ├─ plot.py
│   ├─ resultados.csv
│   ├─ resultados_fork.csv
│   ├─ resultados_fork_raw.csv
│   ├─ resultados_raw.csv
│   ├─ run.log
│   ├─ run.sh
│   ├─ speedup.png
│   ├─ speedup_comparativa.png
│   ├─ speedup_vs_N.png
│   ├─ speedup_vs_N_fork.png
│   ├─ speedup_vs_T.png
│   └─ speedup_vs_T_fork.png
├─ include/
│   ├─ fork_mult.h
│   ├─ llenado.h
│   ├─ matrix_shm.h
│   ├─ memoria14.h
│   └─ pthread_mult.h
```

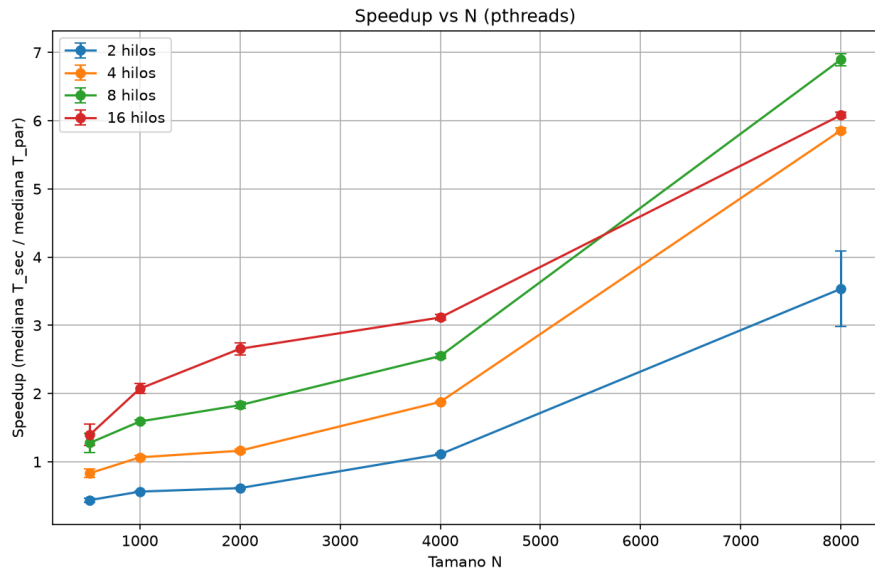


Figura 2. Speedup de pthreads en función del tamaño de la matriz. Cada curva representa una cantidad fija de hilos.

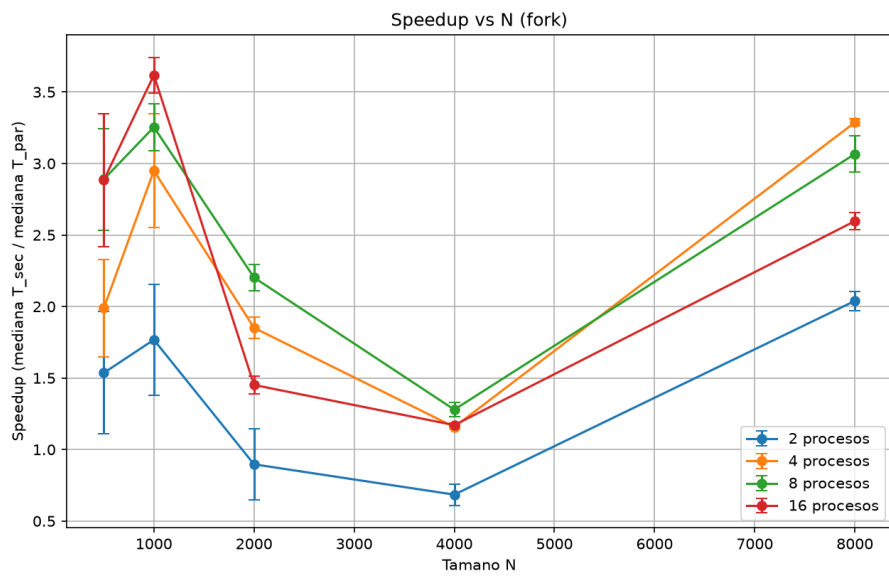


Figura 3. Speedup de fork en función del tamaño de la matriz.

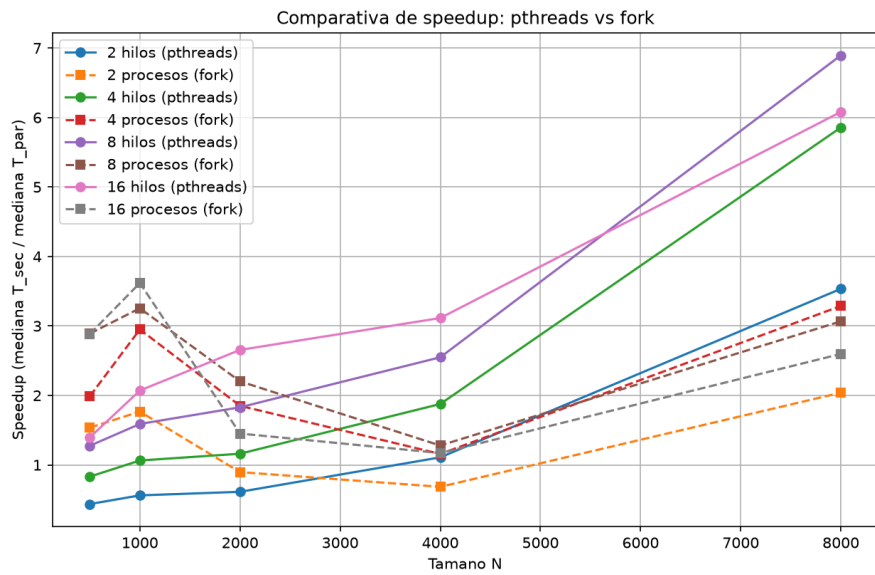


Figura 4. Comparativa de speedup: pthreads vs fork. Las líneas continuas con círculos son pthreads, las líneas punteadas con cuadrados son fork.